

Тема 11. Правила безопасности и аутентификация в Firebase.



Хекслет колледж

Цель занятия:

Научить студентов защищать данные в Firestore через правила безопасности и реализовывать систему регистрации/входа пользователей.

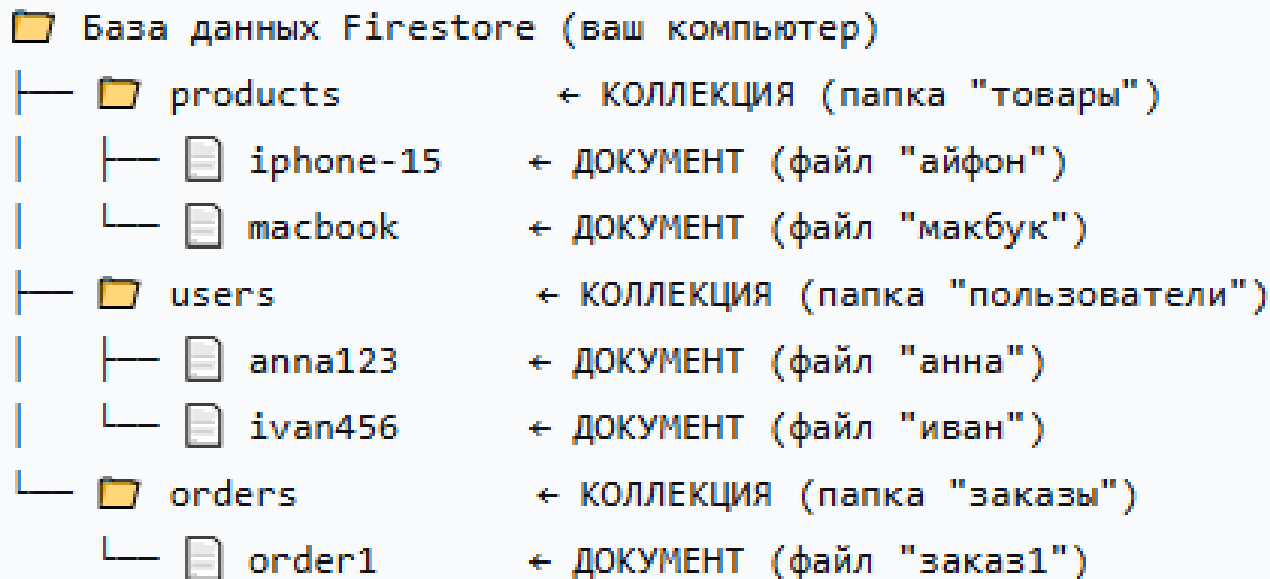
Учебные вопросы:

1. Правила безопасности Firestore
2. Firebase Authentication

1. Правила безопасности Firestore

Что такое правила безопасности?

Firestore устроен как папки на компьютере:



Правила безопасности — это разрешения для каждой папки и файла:

- Кто может открыть эту папку?
- Кто может прочитать этот файл?
- Кто может изменить этот файл?

Термины в правилах:

match /products/{productId} — "Для коллекции 'products'..."

Разбираем:

- products — имя коллекции
- {productId} — переменная, означает "любой документ в этой коллекции"
- productId — это ID документа (например: "iphone-15", "macbook")

allow read, write — "Разрешить читать и писать"

allow read: if condition; // Читать документы

allow write: if condition; //
Создавать/изменять/удалять документы

if condition — это "УСЛОВИЕ"

Пример. "Всегда ДА":

allow read: if true;

Перевод: "Разрешить читать всегда, без условий"

Пример. "Всегда НЕТ":

allow read: if false;

Перевод: "Никогда не разрешать читать"

if request.auth != null — "Если пользователь вошёл"

- request.auth — информация о пользователе
- null — "ничего" (пользователь не вошёл)
- != null — "не равно ничего" (пользователь вошёл)

Примеры правил для интернет-магазина:

Пример 1: Коллекция "products" (товары)

// Для коллекции "products" и любого документа внутри

```
match /products/{productId} {
```

```
// Читать документы могут ВСЕ
```

```
allow read: if true;
```

// Писать (добавлять/изменять/удалять) могут ТОЛЬКО вошедшие пользователи

```
allow write: if request.auth != null;
```

```
}
```

Что это значит:

- Любой человек может смотреть товары 
- Добавить новый товар может только тот, кто вошёл в аккаунт 
- Гость (не вошедший) не может изменить товары 

Пример 2: Коллекция "users" (пользователи)

// Для коллекции "users" и любого документа внутри

match /users/{userId} {

// Читать и писать может ТОЛЬКО владелец этого документа

allow read, write: if request.auth.uid == userId;

}

Что это значит:

- В коллекции users есть документы: "anna123", "ivan456", "petr789"
- У Анны (документ "anna123") есть её данные
- Анна может читать и менять СВОЙ документ "anna123" 
- Анна НЕ может читать документ "ivan456" (данные Ивана) 
- Иван НЕ может менять документ "anna123" (данные Анны) 

Пример 3: Коллекция "orders" (заказы)

// Для коллекции "orders" и любого документа внутри

match /orders/{orderId} {

// Читать документ могут: 1. Пользователь, который создал этот заказ. 2. Администратор

allow read: if

request.auth != null &&

(

resource.data.userId == request.auth.uid ||

request.auth.token.isAdmin == true

);

// Создавать документы могут ТОЛЬКО вошедшие пользователи

allow create: if request.auth != null;

}

Что это значит:

- В коллекции orders есть документы с заказами
- Каждый документ содержит поле userId (чей заказ)
- Пользователь видит только СВОИ заказы 
- Администратор видит ВСЕ заказы 
- Гость не видит НИКАКИХ заказов 
- Любой вошедший может создать новый документ-заказ 

Специальные переменные:

1. request — "Запрос пользователя"

// Что пользователь хочет сделать?

request.method // "read" или "write"

request.time // Когда пытаются

request.auth // Кто пытается

Пример:

// "Разрешить только до 15 марта"

allow write: if request.time < timestamp.date(2026, 3, 15);

request.auth — "Паспорт пользователя"
// Информация о вошедшем пользователе
request.auth.uid // Уникальный ID пользователя
request.auth.token.role // Роль (admin, user, moderator)
request.auth != null // Вошёл ли вообще?

Примеры:

// "Только вошедшие пользователи"
allow read: if request.auth != null;
// "Только пользователь anna123"
allow write: if request.auth.uid == "anna123";
// "Только администраторы"
allow delete: if request.auth.token.role == "admin";

2. resource — "Существующий документ"

// Данные УЖЕ лежащие в базе

resource.data // Все поля документа

resource.data.ownerId // Конкретное поле

Пример:

// Документ в базе:

```
{  
  "title": "iPhone 15",  
  "price": 999,  
  "ownerId": "anna123" ← resource.data.ownerId  
}
```

// Правило:

allow read: if request.auth.uid == resource.data.ownerId;

// "Анна может читать, если она владелец"

Правила
делают ваши данные
безопаснее

request / resource / request.resource

Три ключевые сущности в правилах безопасности Firestore



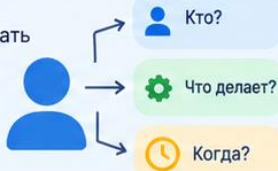
Те же данные.
Больше возможностей!



request

Текущий запрос пользователя

- Кто выполняет действие
- Какое действие хочет сделать
- Когда выполняется запрос
- `request.auth`
- `request.method`
- `request.time`



rules

```
allow read: if request.auth != null;
```



Разрешить чтение только вошедшим пользователям.



resource

Документ, который уже существует в базе

- Текущие данные документа
- Используется при чтении, обновлении и удалении
- `resource.data`
- Например: владелец, цена, статус



Уже в базе

json

```
resource.data = {
  ownerId: 'anna123',
  price: 999
}
```

rules

```
allow read: if request.auth.uid ==
resource.data.ownerId;
```



Пользователь читает только свой документ.



request.resource

Новые данные, которые пытаются записать

- Будущее состояние документа
- Проверка данных перед записью
- Используется при create и update
- `request.resource.data`



Новые данные

json

```
request.resource.data = {
  name: 'iPhone 15',
  price: 999
}
```

rules

```
allow create: if request.resource.data.price > 0;
```



Нельзя сохранить товар с нулевой или отрицательной ценой.

Когда что использовать?



request

→ проверка пользователя и действия



resource

→ проверка текущих данных в базе



request.resource

→ проверка новых данных перед записью



Запомнить

`request` = кто и что делает сейчас

`resource` = что уже хранится в документе

`request.resource` = что хотят сохранить в документе

Три сущности.
Одна безопасная база!

Опасные правила (НИКОГДА так не делайте):

ОПАСНО: Разрешить всё для всех документов
Для коллекции products и ЛЮБОЙ ВЛОЖЕННОСТИ

```
match /products/{document=**} {  
  allow read, write: if true; // ⚠ ОПАСНО!  
}
```

Что значит {document=**}:

- * — один уровень (только документы в коллекции)
- ** — любая вложенность (документы + подколлекции + их документы)

Что случится: Любой человек в интернете сможет:

- Удалить все документы в коллекции products
- Изменить данные в любом документе
- Добавить миллион фейковых документов

Правильный подход к правилам:

Шаг 1: Запретить всё по умолчанию.

Для любой коллекции и любого документа

```
match /{document=**} {  
  allow read, write: if false; // ВСЁ ЗАПРЕЩЕНО  
}
```

Шаг 2: Постепенно разрешать нужное

1. Разрешить всем читать товары

```
match /products/{productId} {  
  allow read: if true;  
  allow write: if false;  
}
```

2. Разрешить пользователям читать свои данные

```
match /users/{userId} {  
  allow read, write: if request.auth.uid == userId;  
}
```

3. Разрешить создавать заказы

```
match /orders/{orderId} {  
  allow create: if request.auth != null;  
  allow read: if request.auth.uid ==  
resource.data.userId;  
}
```

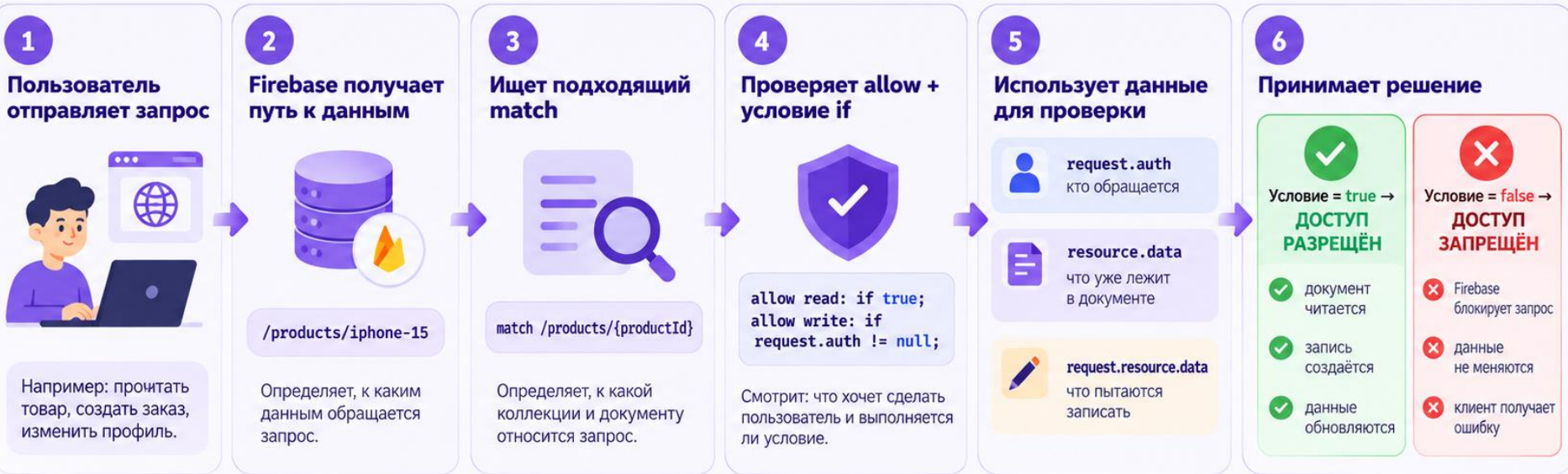
Схема работы правил:

1. Пользователь пытается прочитать документ
- ↓
2. Firebase смотрит: в какой коллекции этот документ?
- ↓
3. Firebase находит правила для этой коллекции
- ↓
4. Проверяет условие после `if`
- ↓
5. Если условие true → разрешает
Если условие false → блокирует

Как работают Security Rules

Как Firebase решает: разрешить запрос или заблокировать

Безопасные
данные.
Лучшие приложения.



Запомните три главных правила:

- 1** Сначала запрещаем всё по умолчанию
- 2** Потом разрешаем только нужные действия
- 3** Правила проверяются при каждом запросе



Итог: Security Rules — это автоматическая проверка доступа к данным в Firestore.

Те же правила.
Более надёжные приложения.



Пример правильных правил для коллекции products

```
match /products/{productId} {
```

```
// 1. Все видят товары
```

```
allow read: if true;
```

```
// 2. Админы управляют товарами
```

```
allow write: if
```

```
  request.auth != null &&
```

```
  request.auth.token.role == "admin";
```

```
// 3. Проверка данных при создании
```

```
allow create: if
```

```
  request.auth != null &&
```

```
  request.auth.token.role == "admin" &&
```

```
  request.resource.data.price > 0 &&
```

```
  request.resource.data.name.size() >= 3 &&
```

```
  request.resource.data.category in ["electronics", "clothing", "books"];
```

```
}
```

Один проект — разные роли — больше возможностей! 🎯

Гость / пользователь / админ

Кто что может делать в приложении с Firebase



Гость

Не авторизован

Посещает приложение без входа в систему.

- 👁 Не вошёл в систему
- ✅ Может просматривать публичные данные
- ❌ Не может изменять данные
- ❌ Не видит личный профиль
- ❌ Не видит свои заказы



Пользователь

Авторизован

Обычный пользователь приложения.

- ✅ Вошёл в систему
- ✅ Может просматривать товары
- ✅ Может читать и менять свой профиль
- ✅ Может создавать заказы
- ✅ Видит только свои заказы
- ❌ Не управляет товарами



Админ

Расширенные права

Управляет приложением и его данными.

- ✅ Вошёл в систему
- ✅ Может просматривать товары
- ✅ Может добавлять, изменять и удалять товары
- ✅ Может видеть все заказы
- ✅ Может управлять данными приложения
- ✅ Имеет расширенные права

Действие / Данные

Гость

Пользователь

Админ

Товары (чтение)



Товары (изменение)



Свой профиль



Чужой профиль



Создание заказа



Свои заказы



Все заказы



Запомнить

Права доступа зависят от того, вошёл ли пользователь в систему и какая у него роль.

Безопасность делает приложение надёжнее!



Как это связано с Security Rules

В Firebase роль определяется по аутентификации и кастомным claims в токене:

```
request.auth == null → гость  
request.auth != null → пользователь  
request.auth.token.role == 'admin' → админ
```



Роли и права проверяются в Security Rules на стороне Firebase, а не в интерфейсе приложения.

Выводы:

Правила нужны чтобы защитить данные от:

- ✗ Чужих глаз (конфиденциальность)
- ✗ Вандализма (удаление/изменение)
- ✗ Спамы (фейковые данные)

3 главных принципа:

- Запрещено по умолчанию — начинай с **if false**
- Разные правила для разных коллекций — товары, пользователи, заказы
- Проверь кто перед тобой — гость, пользователь, админ

✓ Как писать правила:

```
match /коллекция/{документ} {  
  allow read: if условие; // Кто может смотреть  
  allow write: if условие; // Кто может изменять  
}
```

⊘ Никогда не делай:

- allow read, write: if true; // ⚠ Опасно!
- Открытая база = сломают за 5 минут

2. Firebase Authentication

Что такое Firebase Authentication?

Это сервис, который:

- Регистрирует пользователей
- Проверяет логины и пароли
- Помнит, кто вошёл в приложение
- Работает бесплатно. Лимит 3,000 DAU (Daily Active Users) для большинства провайдеров

Настройка в Console

Шаг 1: Включите аутентификацию

Firebase Console → Ваш проект → Build →
Authentication



[Get started]

Шаг 2: Выберите способы входа

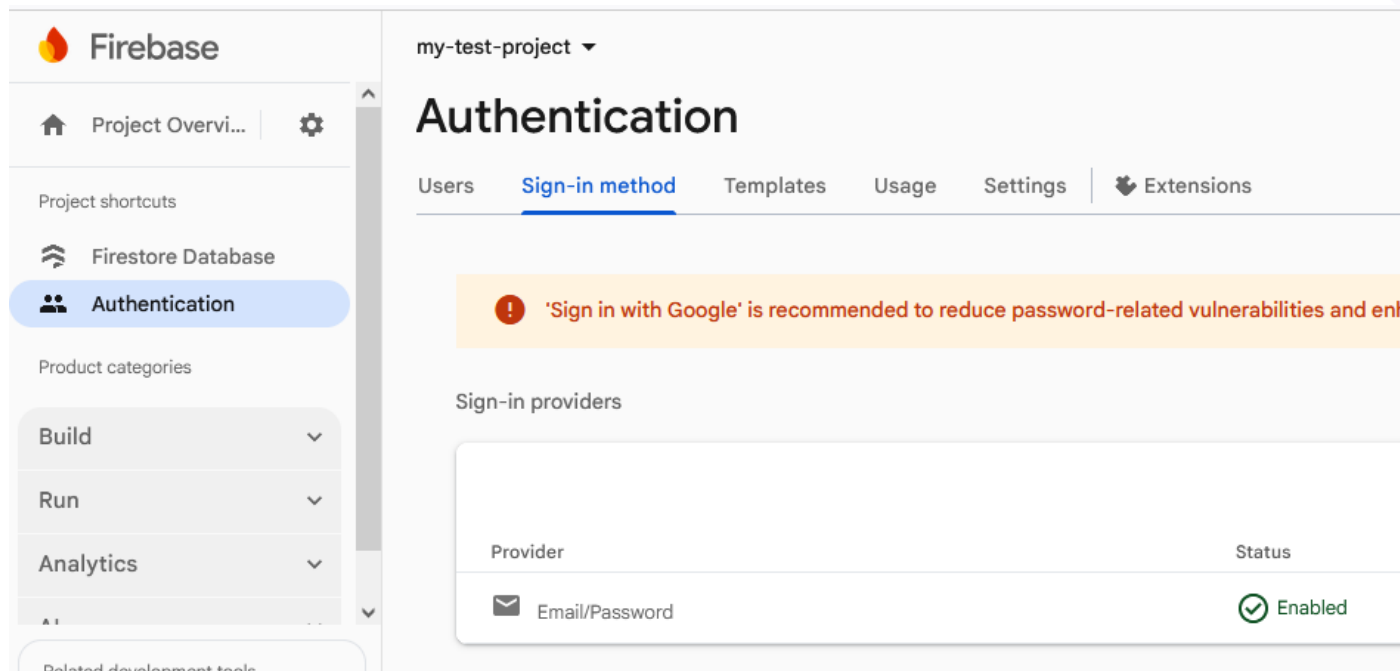
Sign-in method → [Add new provider]

Основные методы:

- Email/Password — по почте и паролю (самый простой)
- Google — вход через аккаунт Google
- Phone — по номеру телефона
- GitHub — для разработчиков

Для начала включите:

- Email/Password → Включить → Сохранить
- Готово! Аутентификация настроена.



Основные методы Firebase Authentication

Настройка и инициализация:

```
<!-- Подключаем Firebase SDK -->
```

```
<script src="https://www.gstatic.com/firebasejs/12.8.0/firebase-auth-compat.js"></script>
```

```
// Инициализация
```

```
const app = firebase.initializeApp(firebaseConfig);
```

```
const auth = firebase.auth(); // ← Главный объект для работы
```

auth — это ваш "ключ" ко всем методам аутентификации. Через него вы регистрируете, входите, выходите.

Auth + Firestore: кто где хранит данные

Authentication хранит данные для входа, Firestore — данные приложения

Два сервиса —
одна цель:
лучшие приложения!



Firebase Authentication

Кто пользователь и как он входит

Здесь хранятся данные для аутентификации пользователя:



uid

Уникальный идентификатор



email

Электронная почта



provider

Способ входа (Email/Password, Google, Apple и др.)



emailVerified

Подтверждена ли почта



Факт входа

Активная сессия пользователя



Основная информация (опционально)
displayName, photoURL

Пример данных в Authentication:

```
{
  uid: "abc123",
  email: "ivan@mail.com",
  provider: "password",
  emailVerified: true,
  displayName: "Иван",
  photoURL: "https://..."
}
```



Пароль не хранится в Firestore!

Пароли безопасно хранятся и управляются Firebase Authentication.

Общий ключ:

UID

Один и тот же uid связывает данные в Auth и Firestore



uid: abc123

Один пользователь —
одни данные
в разных сервисах



Cloud Firestore

Какие данные нужны приложению

Здесь хранятся данные приложения:



Профиль пользователя

Полное имя, город, телефон и др.



Роли и настройки

role, preferences, theme и др.



Корзина / Избранное

Товары в корзине, избранные товары



Заказы

История заказов



Любые другие данные

Посты, комментарии, настройки приложения и т.п.

Пример данных в Firestore:

users / abc123

```
{
  fullName: "Иван Петров",
  city: "Алматы",
  phone: "+7 701 123 45 67",
  role: "user",
  favorites: ["prod_1", "prod_5"],
  createdAt: "2024-05-10"
}
```

orders / order_001

```
{
  uid: "abc123",
  items: ["prod_1", "prod_2"],
  total: 25990,
  status: "created",
  createdAt: "2024-06-15"
}
```



Как это работает

1



Пользователь регистрируется или входит через Auth

Например, по email или через Google

2



Firebase создаёт аккаунт и UID

Пользователю присваивается уникальный uid

3



В Firestore создаётся документ /users/{uid}

Мы сохраняем данные пользователя в коллекции с тем же uid

4



Приложение связывает данные по одному uid

Используем один и тот же uid для работы с данными в обоих сервисах



Запомнить:



Auth = вход и личность пользователя



Firestore = данные приложения



uid = связующее звено



Важно!



Не храните пароли в Firestore

Пароли управляются Firebase Authentication.



Используйте Security Rules для защиты данных

Ограничивайте доступ к данным в Firestore.



Основные методы (для Email/Password)

Регистрация пользователя.

`createUserWithEmailAndPassword(email, password)`

Что делает:

- Создает нового пользователя в системе Firebase Authentication.

Что принимает:

- `auth`, // объект аутентификации (получаем через `firebase.auth()`)
- `"user@example.com"`, // строка, email пользователя
- `"password123"` // строка, пароль (минимум 6 символов)

Что возвращает:

Promise (обещание), который при успехе возвращает объект `UserCredential`:

```
{
  user: {
    uid: "abc123def456...",      // Уникальный ID пользователя
    email: "user@example.com",  // Email пользователя
    emailVerified: false,       // Подтвержден ли email
    displayName: null,          // Имя пользователя (пока null)
    photoURL: null,             // Фото пользователя (пока null)
    metadata: {
      createTime: "2024-01-15T10:30:00Z", // Когда создан
      lastSignInTime: "2024-01-15T10:30:00Z" // Когда последний раз входил
    }
    // ... другие свойства
  },
  providerId: "password",       // Способ аутентификации
  operationType: "signIn"      // Тип операции
}
```

Пример использования:

// 1. Получаем объект аутентификации

```
const auth = firebase.auth();
```

// 2. Вызываем метод

```
auth.createUserWithEmailAndPassword("ivan@mail.ru", "mypassword")
```

```
.then((userCredential) => {
```

// 3. Работаем с результатом

```
const user = userCredential.user;
```

```
console.log("Создан пользователь с ID:", user.uid);
```

```
console.log("Email:", user.email);
```

```
})
```

```
.catch((error) => {
```

// 4. Обрабатываем ошибки

```
console.error("Ошибка:", error.message);
```

```
});
```

Что происходит внутри:

- Firebase проверяет, нет ли уже пользователя с таким email
- Хеширует пароль (превращает в нечитаемую строку)
- Сохраняет данные в своей защищенной базе
- Автоматически авторизует пользователя
- Возвращает информацию о созданном пользователе

Вход пользователя

signInWithEmailAndPassword(email, password)

Что делает:

- Авторизует существующего пользователя (вход в систему).

Что принимает:

То же самое, что и

`createUserWithEmailAndPassword`:

- объект аутентификации
- email (строка)
- пароль (строка)

Что возвращает:

- Тот же `UserCredential` объект, что и при регистрации.

Пример использования:

```
const auth = firebase.auth();

auth.signInWithEmailAndPassword("ivan@mail.ru", "mypassword")
  .then((userCredential) => {
    const user = userCredential.user;
    // После успешного входа:
    console.log("Добро пожаловать,", user.email);
    console.log("Ваш ID:", user.uid);
  })
  .catch((error) => {
    if (error.code === "auth/user-not-found") {
      alert("Пользователь не найден!");
    } else if (error.code === "auth/wrong-password") {
      alert("Неверный пароль!");
    }
  });
```

Что происходит внутри:

- Firebase находит пользователя по email
- Сравнивает хеши паролей (не сами пароли!)
- Если совпадает — создает сессию
- Возвращает данные пользователя
- Запоминает пользователя в браузере

Важно: После успешного входа пользователь остается авторизованным даже если:

- Закрывать вкладку браузера
- Перезагрузить страницу
- Выключить и включить компьютер
- Пока пользователь явно не выйдет (signOut) или не очистит куки.

Выход пользователя

`signOut()`

Что делает:

- Завершает текущую сессию пользователя (выход из системы).

Что принимает:

- Ничего не принимает, или можно передать объект аутентификации:

//Пример:

```
const auth = firebase.auth();  
auth.signOut();
```


Что возвращает:

- Promise, который завершается успешно когда:
 - Сессия завершена
 - Все связанные токены удалены

Пример использования:

```
const auth = firebase.auth();
```

```
function logout() {  
  auth.signOut()  
    .then(() => {  
      console.log("Выход выполнен успешно");  
  
      // Обновляем интерфейс  
      document.getElementById("user-menu").style.display = "none";  
      document.getElementById("login-form").style.display = "block";  
    })  
    .catch((error) => {  
      console.error("Ошибка при выходе:", error);  
    });  
}
```

```
// Кнопка выхода
```

```
document.getElementById("logout-btn").addEventListener("click", logout);
```

Что происходит внутри:

- Удаляет токены аутентификации из `localStorage/sessionStorage`
- Сообщает Firebase серверу о завершении сессии
- `onAuthStateChanged` получает `null` (нет пользователя)

Отслеживание состояния пользователя

`onAuthStateChanged(callback)`

Что делает:

- Слушает (подписывается) на изменения состояния аутентификации.

Что принимает:

- Функцию-обработчик (callback), которая принимает один параметр:

```
onAuthStateChanged((user) => {  
  // user - объект пользователя или null  
});
```

Что возвращает:

Функцию для отписки (unsubscribe):

```
const unsubscribe =  
auth.onAuthStateChanged((user) => {  
  // ...  
});
```

```
// Когда нужно перестать слушать:  
unsubscribe();
```

Пример использования:

```
const auth = firebase.auth();
```

```
// Подписываемся на изменения
```

```
const unsubscribe = auth.onAuthStateChanged((user) => {  
  console.log("Состояние изменилось!");
```

```
  if (user) {
```

```
    // Есть пользователь (вошел или уже был авторизован)
```

```
    console.log("Пользователь:", user.email);
```

```
    showUserInterface(user);
```

```
  } else {
```

```
    // Нет пользователя (вышел или никогда не входил)
```

```
    console.log("Пользователь не авторизован");
```

```
    showLoginInterface();
```

```
  }
```

```
});
```

Эта функция вызывается Firebase автоматически:

- При загрузке страницы
- После успешного входа (`signInWithEmailAndPassword`)
- После выхода (`signOut`)
- При истечении сессии
- При ошибках аутентификации

Зачем нужно отслеживать состояние авторизации:

- Чтобы не просить логин/пароль каждый раз
- Чтобы данные не терялись при обновлении страницы
- Чтобы показывать разные интерфейсы

Жизненный цикл пользователя

От регистрации до выхода — как работает пользователь в приложении с Firebase



Один пользователь —
много возможностей

1 Регистрация

Пользователь создаёт
аккаунт (email/пароль
или другой способ)

Registration form UI showing a user icon, email input field with 'student@example.com', password input field with masked characters, and a 'Зарегистрироваться' button.

```
// Регистрация (email/пароль)
auth.createUserWithEmailAndPassword(
  email, password
)
```

2 Вход

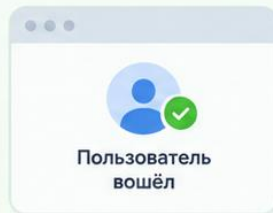
Пользователь входит
в свой аккаунт

Login form UI showing a user icon, email input field with 'student@example.com', password input field with masked characters, and a 'Войти' button.

```
// Вход (email/пароль)
auth.signInWithEmailAndPassword(
  email, password
)
```

3 Отслеживание статуса

Приложение узнаёт,
вошёл ли пользователь



```
// Отслеживание авторизации
auth.onAuthStateChanged(user => {
  if (user) {
    // пользователь вошёл
  } else {
    // пользователь вышел
  }
})
```

4 Работа с данными

Пользователь читает
и изменяет данные
в Firestore (по правилам
безопасности)



Профиль
Заказы
Избранное
...

```
// Пример: получить профиль
const uid = auth.currentUser.uid;
db.collection('users')
  .doc(uid)
  .get();
```

5 Выход

Пользователь выходит
из аккаунта



Выйти

```
// Выход
auth.signOut();
```

Пользователь может снова войти



Безопасно

Firebase Authentication защищает
учётные данные пользователя



Удобно

Готовые методы для регистрации,
входа и выхода



Гибко

Поддержка email/пароль, Google, GitHub,
Apple и других провайдеров

Больше, чем просто
вход в приложение

Работа с профилем пользователя

Получение текущего пользователя:

auth.currentUser

Что делает:

- Возвращает текущего авторизованного пользователя.

Что принимает:

- Ничего не принимает.

Что возвращает:

- Объект пользователя, если пользователь авторизован
null, если пользователь не авторизован

Пример использования:

```
const auth = firebase.auth();
```

```
// Проверка в любом месте кода
```

```
function checkIfLoggedIn() {
```

```
  const user = auth.currentUser;
```

```
  if (user) {
```

```
    console.log("Пользователь авторизован:", user.email);
```

```
    return true;
```

```
  } else {
```

```
    console.log("Пользователь не авторизован");
```

```
    return false;
```

```
  }
```

```
}
```

Использование перед важными действиями

```
function addToCart(productId) {  
  if (!auth.currentUser) {  
    alert("Войдите, чтобы добавлять товары в корзину!");  
    return;  
  }  
  
  // Пользователь есть, можно добавлять  
  const userId = auth.currentUser.uid;  
  // ... добавляем товар  
}
```

Важные моменты:

- `currentUser` обновляется автоматически
- Может быть `null` сразу после загрузки страницы (пока `onAuthStateChanged` не сработал)
- Используйте вместе с `onAuthStateChanged` для надежности

Обновление профиля

user.updateProfile(profileData)

Что делает:

- Обновляет базовую информацию о пользователе (имя и фото).

Что принимает:

- Объект с полями для обновления:

```
{  
  displayName: "Новое имя", // строка (не обязательно)  
  photoURL: "https://..." // строка, URL фото (не обязательно)  
}
```

Что возвращает:

- Promise, который завершается когда профиль обновлен.

Пример использования:

```
const auth = firebase.auth();
```

```
function updateUserProfile() {
```

```
  // Получаем текущего пользователя
```

```
  const user = auth.currentUser;
```

```
  if (!user) {
```

```
    alert("Войдите, чтобы обновить профиль!");
```

```
    return;
```

```
  }
```

```
  // Готовим данные для обновления
```

```
  const newName = document.getElementById("name-input").value;
```

```
  const newPhoto = document.getElementById("photo-input").value;
```

```
  const profileUpdates = {
```

```
    displayName: newName || user.displayName, // если пусто - оставить старое
```

```
    photoURL: newPhoto || user.photoURL
```

```
  };
```

```
// Вызываем метод
user.updateProfile(profileUpdates)
  .then(() => {
    console.log("Профиль успешно обновлен!");

    // Обновляем интерфейс
    document.getElementById("user-name").textContent = user.displayName;

    // Проверяем результат
    console.log("Новое имя:", user.displayName);
    console.log("Новое фото:", user.photoURL);
  })
  .catch((error) => {
    console.error("Ошибка обновления:", error);
  });
}
```


Что НЕЛЬЗЯ обновить через `updateProfile`:

- Email (используйте `user.updateEmail()`)
- Пароль (используйте `user.updatePassword()`)
- UID (никогда не меняется)
- Email verification status

Что происходит при обновлении:

- Firebase обновляет данные в своей базе
- НЕ обновляет `auth.currentUser` сразу (только после перезагрузки страницы)
- Что бы увидеть изменения сразу используйте `user.reload()`

Сводная таблица методов

Метод	Принимает	Возвращает	Когда использовать
<code>createUserWithEmailAndPassword()</code>	email, password	UserCredential	При регистрации нового пользователя
<code>signInWithEmailAndPassword()</code>	email, password	UserCredential	При входе существующего пользователя
<code>signOut()</code>	ничего	Promise	При выходе из системы
<code>onAuthStateChanged()</code>	callback-функцию	функция отписки	При загрузке страницы, для отслеживания состояния
<code>auth.currentUser</code>	ничего	User или null	В любой момент для проверки авторизации
<code>user.updateProfile()</code>	объект с displayName/photoURL	Promise	При изменении имени или фото пользователя

Выводы по теме:

Firebase Authentication. ДЛЯ ЧЕГО:

- Регистрация новых пользователей
- Вход в систему
- Выход из системы
- Запоминание пользователя

ПОДДЕРЖИВАЕТ:

- Email/пароль (самое простое)
- Google, Facebook, GitHub
- Телефон (SMS)
- Анонимный вход

КАК РАБОТАЕТ:

- Auth → хранит email и пароль
- Firestore → хранит имя, адрес, профиль
- Один ID на обоих сервисах

БЕЗОПАСНОСТЬ Firebase:

- Сначала всё запретить!
- Потом разрешить нужное
- Каждый пользователь = только свои данные

Контрольные вопросы:

- Что делает Firebase Authentication?
- Как зарегистрировать пользователя?
- Как войти в систему?
- Как выйти из системы?
- Как проверить, вошёл ли пользователь?
- Где хранятся логин/пароль?
- Где хранить имя и адрес пользователя?
- Какие правила безопасности самые важные?
- Что будет при обновлении страницы?

Документация Firebase:

<https://firebase.google.com/docs/firestore/security/get-started?hl=ru>

Правила в песочнице:

<https://firebase.google.com/docs/rules/simulator>

Примеры правил:

<https://firebase.google.com/docs/firestore/security/rules-recipes>

хекслет колледж

@HEXLY.KZ